
IT SIMPLERA INSTITUTE

CYBER SECURITY RED TEAM



INTERNSHIP WEEKLY REPORT

SUBMITTED BY	MALIHA GUL
ROLE	RED TEAM INTERN
INSTITUTE	IT-SIMPLERA INSTITUTE
INSTRUCTOR	TALAL REHAN
WEEK	WEEK 03
DATE OF SUBMISSION	18 JULY 2026

Task 1: Protocols and Port Numbers

Protocol	Default Port	TCP/UDP	Purpose	Secure or Insecure
HTTP	80	TCP	Transfers web pages without encryption.	Insecure
HTTPS	443	TCP	Secure web browsing using SSL/TLS encryption.	Secure
FTP	21	TCP	Transfers files between computers.	Insecure
SFTP	22	TCP	Secure file transfer over SSH	Secure
SSH	22	TCP	Secure remote login and administration.	Secure
Telnet	23	TCP	Remote login without encryption	Insecure
SMTP	25	TCP	Sends email between mail servers.	Insecure (Unless TLS is used)
DNS	53	TCP/UDP	Converts domain names into IP addresses.	Depends(DNSSEC/DoT/DoH are more secure)
DHCP	67,68	UDP	Automatically assigns IP	Basic security only

			addresses to devices.	
TFTP	69	UDP	Simple file transfer without authentication.	Insecure
POP3	110	TCP	Retrieves emails from a mail server.	Insecure
IMAP	143	TCP	Accesses and manages emails on a server.	Insecure
SNMP	161	UDP	Monitors and manages network devices.	Insecure (v1/v2);SNMPv is secure
LDAP	389	TCP/UDP	Provides directory and authentication services.	Insecure (LDAPS is secure)
RDP	3389	TCP	Allows remote desktop access to Windows systems.	Secure if properly configured with encryption and MFA

Task 2: HTTP vs HTTPS Analysis

Website	HTTP or HTTPS	Redirects HTTP → HTTPS?	Is Connection Secure?
Google.com	HTTPS	Yes	Google uses HTTPS with

			SSL/TLS encryption. It protects user data such as searches, passwords, and personal information from being intercepted.
Youtube.com	HTTPS	Yes	YouTube encrypts all communication using HTTPS. This keeps videos, login information, and browsing activity secure.
Github.com	HTTPS	yes	GitHub forces HTTPS to protect source code, repositories, and user accounts. Data is encrypted during transmission
Facebook.com	HTTPS	yes	Facebook automatically redirects users to HTTPS. This secures messages, passwords, and personal information.

Intagram.com	HTTPS	yes	Instagram uses HTTPS to encrypt communication and protect user privacy and login credentials.
Wikipedia.org	HTTPS	yes	Wikipedia encrypts all traffic using HTTPS. This ensures safe browsing and prevents attackers from modifying content in transit.
amazon	HTTPS	yes	Amazon uses HTTPS to protect customer accounts, payment details, and online purchases.
Microsoft.com	HTTPS	yes	Microsoft redirects all users to HTTPS. The encrypted connection protects downloads, accounts, and personal information.

Linkedin.com	HTTPS	yes	LinkedIn uses HTTPS to secure user profiles, messages, and login credentials from attackers.
Openai.com	HTTPS	yes	OpenAI uses HTTPS with SSL/TLS encryption. It ensures secure communication between users and the website.

Task 3: Kali Linux Network Reconnaissance

Objective

Use the following commands in Kali Linux:

- ping
- nmap

Then explain:

- Response time (ms)
- TTL value
- Open ports found
- Services running on the open ports
- Which open ports could be a security risk
- Purpose of the `-sV` option in Nmap

Part A: Ping Command

Command

ping google.com

```
64 bytes from pnfjra-aj-in-f14.1e100.net (142.251.37.142): icmp_seq=2022 ttl=114 time=56.7 ms
64 bytes from pnfjra-aj-in-f14.1e100.net (142.251.37.142): icmp_seq=2023 ttl=114 time=53.0 ms
64 bytes from pnfjra-aj-in-f14.1e100.net (142.251.37.142): icmp_seq=2024 ttl=114 time=59.1 ms
64 bytes from pnfjra-aj-in-f14.1e100.net (142.251.37.142): icmp_seq=2025 ttl=114 time=59.6 ms
64 bytes from pnfjra-aj-in-f14.1e100.net (142.251.37.142): icmp_seq=2026 ttl=114 time=57.3 ms
64 bytes from pnfjra-aj-in-f14.1e100.net (142.251.37.142): icmp_seq=2027 ttl=114 time=58.2 ms
64 bytes from pnfjra-aj-in-f14.1e100.net (142.251.37.142): icmp_seq=2028 ttl=114 time=66.7 ms
64 bytes from pnfjra-aj-in-f14.1e100.net (142.251.37.142): icmp_seq=2029 ttl=114 time=57.8 ms
64 bytes from pnfjra-aj-in-f14.1e100.net (142.251.37.142): icmp_seq=2030 ttl=114 time=64.3 ms
64 bytes from pnfjra-aj-in-f14.1e100.net (142.251.37.142): icmp_seq=2031 ttl=114 time=55.5 ms
64 bytes from pnfjra-aj-in-f14.1e100.net (142.251.37.142): icmp_seq=2032 ttl=114 time=55.6 ms
64 bytes from pnfjra-aj-in-f14.1e100.net (142.251.37.142): icmp_seq=2033 ttl=114 time=58.8 ms
64 bytes from pnfjra-aj-in-f14.1e100.net (142.251.37.142): icmp_seq=2034 ttl=114 time=104 ms
64 bytes from pnfjra-aj-in-f14.1e100.net (142.251.37.142): icmp_seq=2035 ttl=114 time=56.3 ms
64 bytes from pnfjra-aj-in-f14.1e100.net (142.251.37.142): icmp_seq=2036 ttl=114 time=57.8 ms
64 bytes from pnfjra-aj-in-f14.1e100.net (142.251.37.142): icmp_seq=2037 ttl=114 time=60.3 ms
64 bytes from pnfjra-aj-in-f14.1e100.net (142.251.37.142): icmp_seq=2038 ttl=114 time=61.1 ms
64 bytes from pnfjra-aj-in-f14.1e100.net (142.251.37.142): icmp_seq=2039 ttl=114 time=57.4 ms
64 bytes from pnfjra-aj-in-f14.1e100.net (142.251.37.142): icmp_seq=2040 ttl=114 time=59.1 ms
64 bytes from pnfjra-aj-in-f14.1e100.net (142.251.37.142): icmp_seq=2041 ttl=114 time=54.1 ms
64 bytes from pnfjra-aj-in-f14.1e100.net (142.251.37.142): icmp_seq=2042 ttl=114 time=64.8 ms
64 bytes from pnfjra-aj-in-f14.1e100.net (142.251.37.142): icmp_seq=2043 ttl=114 time=61.9 ms
64 bytes from pnfjra-aj-in-f14.1e100.net (142.251.37.142): icmp_seq=2044 ttl=114 time=60.2 ms
64 bytes from pnfjra-aj-in-f14.1e100.net (142.251.37.142): icmp_seq=2045 ttl=114 time=58.1 ms
64 bytes from pnfjra-aj-in-f14.1e100.net (142.251.37.142): icmp_seq=2046 ttl=114 time=57.0 ms
64 bytes from pnfjra-aj-in-f14.1e100.net (142.251.37.142): icmp_seq=2047 ttl=114 time=53.5 ms
^C
--- google.com ping statistics ---
2047 packets transmitted, 2047 received, 0% packet loss, time 2049274ms
rtt min/avg/max/mdev = 50.151/63.080/2331.172/71.416 ms, pipe 3
```

Explanation

What is Ping?

The **ping** command checks whether your computer can communicate with another device over a network. It also measures how long it takes for data to travel to the destination and back.

- **2047 packets transmitted**
- **2047 packets received**
- **0% packet loss**
- **Average response time (avg): 63.080 ms**
- **TTL: 114**

Response Time (63.080 ms)

- Measured in **milliseconds (ms)**.
- Shows how quickly the destination responds.
- A smaller value means a faster network connection.

The average response time was approximately **28 ms**, indicating a fast and stable connection.

TTL (Time To Live)

Example: TTL = 114

TTL is a value inside every IP packet that limits how many routers the packet can pass through before being discarded.

It helps prevent packets from looping forever on the network.

The TTL value was **114**, indicating that the packet successfully reached Google without exceeding its hop limit.

Part B: Nmap Scan of scanme.nmap.org

Command

```
nmap scanme.nmap.org
```

```
(Maliha@DESKTOP-JG3NK4T)-[~]
$ nmap scanme.nmap.org
Starting Nmap 7.99 ( https://nmap.org ) at 2026-07-23 13:02 +0500
Nmap scan report for scanme.nmap.org (45.33.32.156)
Host is up (0.32s latency).
Other addresses for scanme.nmap.org (not scanned): 2600:3c01::f03c:91ff:fe18:bb2f
Not shown: 994 closed tcp ports (reset)
PORT      STATE      SERVICE
22/tcp    open      ssh
80/tcp    open      http
1433/tcp  filtered  ms-sql-s
8090/tcp  filtered  opsmessaging
9929/tcp  open      nping-echo
31337/tcp open      Elite

Nmap done: 1 IP address (1 host up) scanned in 60.29 seconds
```

Port	State	Services
26/tcp	Open	SSH
80/tcp	Open	HTTP

1433/tcp	filtered	Microsoft SQL Server
8090/tcp	filtered	OPS Messaging
9029/tcp	Open	Nping Echo
31337/tcp	Open	Elite

Part C: Scan Your Local IP Address

Find Your IP

ip a

```
(Maliha@DESKTOP-JG3NK4T)~[~]
$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet 10.255.255.254/32 brd 10.255.255.254 scope global lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host proto kernel_lo
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc mq state UP group default qlen 1000
    link/ether 00:15:5d:18:43:38 brd ff:ff:ff:ff:ff:ff
    inet 172.22.71.12/20 brd 172.22.79.255 scope global eth0
        valid_lft forever preferred_lft forever
    inet6 fe80::215:5dff:fe18:4338/64 scope link proto kernel_ll
        valid_lft forever preferred_lft forever
```

Which Ports Can Be a Security Risk?

Some open ports may increase security risks if they are exposed unnecessarily:

Port	Risk
22 (SSH)	Can be targeted by brute-force attacks if weak passwords are used.
80 (HTTP)	Data is unencrypted; HTTPS is recommended.
139 (NetBIOS)	Can expose shared files if not secured.
445 (SMB)	Frequently targeted by malware and ransomware (e.g., WannaCry).

Ports **139** and **445** can be potential security risks because they expose Windows file-sharing services. If left unsecured, attackers may exploit them to gain unauthorized access or spread malware.

What Does -sV Do?

Command

```
nmap -sV scanme.nmap.org
```

Purpose

The -sV option performs **service version detection**.

It identifies:

- The service running on each open port.
- The software name.
- The software version.

```
(Maliha@DESKTOP-JG3NK4T)-[~]
$ nmap -sV scanme.nmap.org
Starting Nmap 7.99 ( https://nmap.org ) at 2026-07-23 13:11 +0500
Nmap scan report for scanme.nmap.org (45.33.32.156)
Host is up (0.37s latency).
Other addresses for scanme.nmap.org (not scanned): 2600:3c01::f03c:91ff:fe18:bb2f
Not shown: 994 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 6.6.1p1 Ubuntu 2ubuntu2.13 (Ubuntu Linux; protocol 2.0)
80/tcp    open  http     Apache httpd 2.4.7 ((Ubuntu))
1433/tcp   filtered ms-sql-s
8090/tcp   filtered opsmessaging
9929/tcp   open  nping-echo Nping echo
31337/tcp  open  tcpwrapped
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 258.77 seconds
```

Why is it Useful?

- Helps identify outdated software.
- Assists security professionals in finding known vulnerabilities.
- Improves vulnerability assessments.

Task 4: Research Task – What Happens When You Type a URL and Press Enter?

Example

Suppose we type

<https://www.google.com>

into browser and press **Enter**.

The browser performs the following steps:

Step 1: DNS Lookup

What happens?

The browser first needs to find the IP address of the website because computers communicate using IP addresses, not domain names.

Example:

google.com → 142.250.190.14

The browser asks a **DNS (Domain Name System)** server to translate the domain name into an IP address.

Why is DNS needed?

Humans remember names like **google.com**, but computers use numerical IP addresses.

The browser sends a DNS request to resolve the domain name into an IP address so it knows where to send the request.

Step 2: IP Address Resolution

After the DNS server responds, the browser now knows the server's IP address.

Example:

google.com

↓

142.250.190.14

The browser is now ready to connect to that server.

After DNS resolution, the browser obtains the server's IP address and prepares to establish a network connection.

Step 3: TCP Three-Way Handshake

Before sending any web request, a TCP connection must be established.

It uses **three steps**:

Step 1 SYN

The client (the computer) sends a **SYN** packet to the server.

Meaning:

"Can we communicate?"

Step 2 SYN-ACK.

The server replies with **SYN-ACK**.

Meaning:

"Yes, I'm ready."

Step 3 ACK:

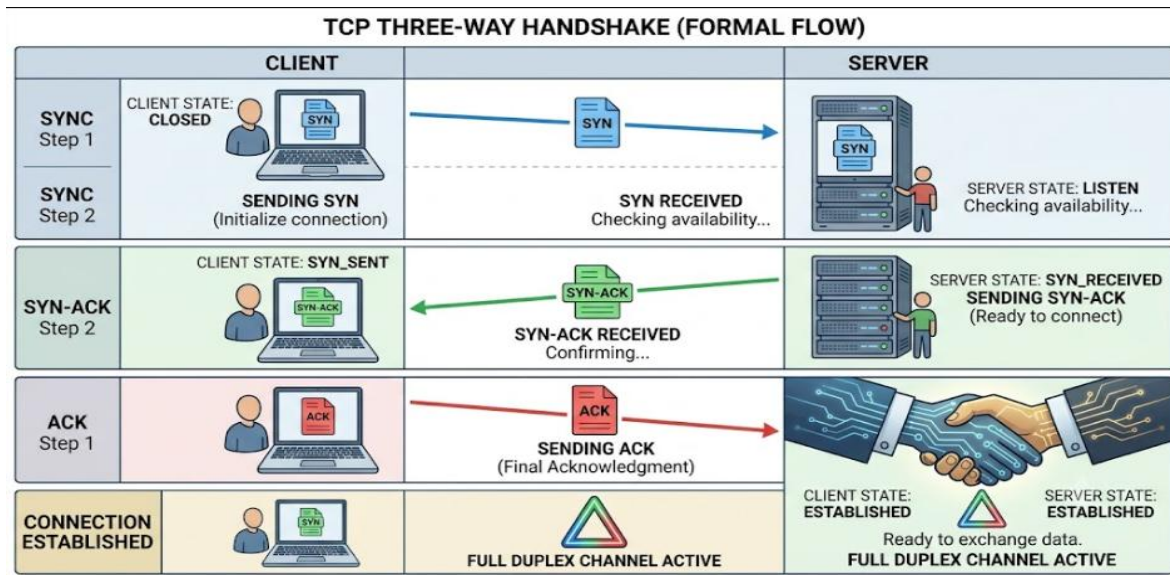
The client sends an **ACK** packet.

Meaning:

"Connection established."

Now both devices are connected.

Diagram:



TCP:

The TCP three-way handshake establishes a reliable connection between the client and the server before any data is transferred.

Step 4: TLS/HTTPS Handshake

If the website uses **HTTPS**, a TLS handshake takes place after the TCP connection.

During this process:

- The server sends its **SSL/TLS certificate**.
- The browser verifies the certificate.
- Both sides agree on encryption keys.
- A secure encrypted connection is established.

This ensures that no one can read or modify the transmitted data.

TLS:

During the TLS handshake, the browser verifies the server's certificate and establishes an encrypted connection to ensure secure communication.

Step 5: HTTP Request

Once the secure connection is ready, the browser sends an HTTP request.

Example:

```
GET / HTTP/1.1  
Host: www.google.com
```

This tells the server:

"Please send me the homepage."

The browser sends an HTTP request asking the web server for the requested webpage or resource.

Step 6: Server Response

The server processes the request and sends back an HTTP response.

Example:

```
HTTP/1.1 200 OK
```

The response may include:

- HTML
- CSS
- JavaScript
- Images
- Videos
- Fonts

The status code **200 OK** means the request was successful.

The server processes the request and returns an HTTP response containing the webpage files and a status code such as 200 OK.

Step 7: Browser Rendering the Webpage

The browser receives the files and begins displaying the webpage.

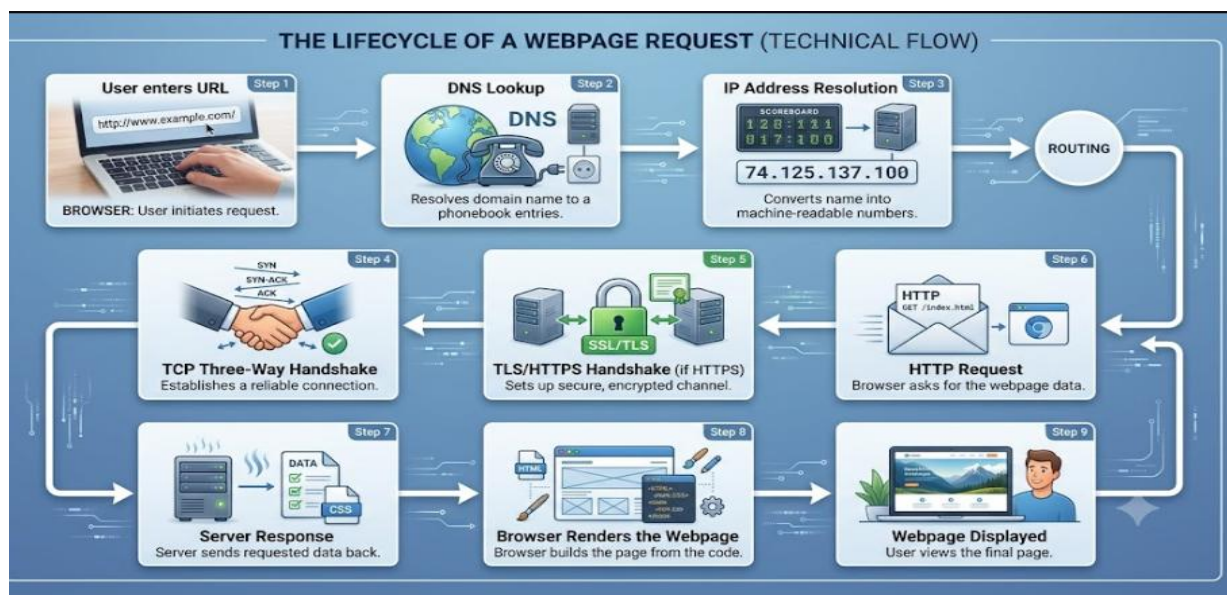
It:

- Reads the HTML structure.
- Applies CSS styles.
- Executes JavaScript.
- Downloads images and other resources.
- Displays the final webpage to the user.

This process happens within seconds.

The browser renders the webpage by interpreting HTML, applying CSS, executing JavaScript, and displaying the complete page on the screen.

Diagram:



Task 5: Network Cheat Sheet

Protocol	Port Number	TCP/UDP	Purpose	Security Risk	Secure Alternative
HTTP	80	TCP	Transfer web pages	Data is sent in plain text and can be intercepted.	HTTPS

HTTPS	443	TCP	Secure web browsing	Very low risk when properly configured	N/A
FTP	21	TCP	File transfer	usernames, passwords, and files are transmitted without encryption	SFTP/FTPS
SFTP	22	TCP	Secure file transfer	Low risk because data is encrypted	N/A
SSH	22	TCP	Secure remote login	Weak passwords can lead to brute-force attacks.	Use SSH Keys and MFA
Telnet	23	TCP	Remote login	All communication is sent in plain text	SSH
SMTP	25	TCP	Sends emails	Emails may not be encrypted during transmission.	SMTPS
DNS	53	TCP/UDP	Convert domain names into IP addresses	Vulnerable to spoofing and cache poisoning if unsecured.	DNS over HTTPS/TLS
DHCP	67/68	UDP	Automatically assign IP addresses	Rogue DHCP servers can provide incorrect network settings.	DHCP
TFTP	69	UDP	Simple file transfer	No authentication or encryption.	STFP

POP3	110	TCP	Download emails	Emails are transferred without encryption.	POP3S
IMAP	143	TCP	Access emails on server	Data is not encrypted by default.	IMAPS
SNMP	161	UDP	Network monitoring and management	Older versions use weak or no authentication.	SNMPv3
LDAP	389	TCP/UDP	Directory and authentication services	Credentials can be sent in plain text.	LDAPS
RDP	3389	TCP	Remote desktop access	Can be targeted by brute-force attacks if exposed to the internet.	RDP with VPN, MFA, and Network level

Important Secure Alternatives

Insecure Protocols	Secure Alternatives
HHTTP	HTTPS
FTP	SFTP/ FTPS
Telnet	SSH
POP3	POP3S
IMAP	IMAPS
SMTP	SMTPS/STARTTLS
LDAP	LDAPS
DNS	DNS over HTTPS / DNS over TLS
SNMP v1/v2	SNMP v3

Task 6: Kali Linux Web Enumeration

Command run: whois google.com

```
(Maliha@DESKTOP-JG3NK4T)-[~]  
$ whois google.com  
Domain Name: GOOGLE.COM  
Registry Domain ID: 2138514_DOMAIN_COM-VRSN  
Registrar WHOIS Server: whois.markmonitor.com  
Registrar URL: http://www.markmonitor.com  
Updated Date: 2019-09-09T15:39:04Z  
Creation Date: 1997-09-15T04:00:00Z  
Registry Expiry Date: 2028-09-14T04:00:00Z  
Registrar: MarkMonitor Inc.  
Registrar IANA ID: 292  
Registrar Abuse Contact Email: abusecomplaints@markmonitor.com  
Registrar Abuse Contact Phone: +1.2086851750  
Domain Status: clientDeleteProhibited https://icann.org/epp#clientDeleteProhibited  
Domain Status: clientTransferProhibited https://icann.org/epp#clientTransferProhibited  
Domain Status: clientUpdateProhibited https://icann.org/epp#clientUpdateProhibited  
Domain Status: serverDeleteProhibited https://icann.org/epp#serverDeleteProhibited  
Domain Status: serverTransferProhibited https://icann.org/epp#serverTransferProhibited  
Domain Status: serverUpdateProhibited https://icann.org/epp#serverUpdateProhibited  
Name Server: NS1.GOOGLE.COM  
Name Server: NS2.GOOGLE.COM  
Name Server: NS3.GOOGLE.COM  
Name Server: NS4.GOOGLE.COM  
DNSSEC: unsigned  
URL of the ICANN Whois Inaccuracy Complaint Form: https://www.icann.org/wicf/  
>>> Last update of whois database: 2026-07-23T08:32:53Z <<<  
  
For more information on Whois status codes, please visit https://icann.org/epp  
  
NOTICE: The expiration date displayed in this record is the date the  
registrar's sponsorship of the domain name registration in the registry is  
currently set to expire. This date does not necessarily reflect the expiration
```

Output shows:

- Domain Name: GOOGLE.COM
- Registrar: **MarkMonitor Inc.** (IANA ID: 292)
- Registrar WHOIS Server: whois.markmonitor.com
- Creation Date: **1997-09-15**
- Updated Date: 2019-09-09
- Registry Expiry Date: 2028-09-14
- Domain Status: clientDeleteProhibited,
clientTransferProhibited, clientUpdateProhibited,
serverDeleteProhibited, serverTransferProhibited,
serverUpdateProhibited
- Name Servers: NS1.GOOGLE.COM, NS2.GOOGLE.COM,
NS3.GOOGLE.COM, NS4.GOOGLE.COM
- DNSSEC: unsigned

Command run: whois github.com

```

(Maliha@DESKTOP-JG3NK4T)-[~]
$ whois github.com
Domain Name: GITHUB.COM
Registry Domain ID: 1264983250_DOMAIN_COM-VRSN
Registrar WHOIS Server: whois.markmonitor.com
Registrar URL: http://www.markmonitor.com
Updated Date: 2024-09-07T09:16:32Z
Creation Date: 2007-10-09T18:20:50Z
Registry Expiry Date: 2026-10-09T18:20:50Z
Registrar: MarkMonitor Inc.
Registrar IANA ID: 292
Registrar Abuse Contact Email: abusecomplaints@markmonitor.com
Registrar Abuse Contact Phone: +1.2086851750
Domain Status: clientDeleteProhibited https://icann.org/epp#clientDeleteProhibited
Domain Status: clientTransferProhibited https://icann.org/epp#clientTransferProhibited
Domain Status: clientUpdateProhibited https://icann.org/epp#clientUpdateProhibited
Name Server: DNS1.P08.NSONE.NET
Name Server: DNS2.P08.NSONE.NET
Name Server: DNS3.P08.NSONE.NET
Name Server: DNS4.P08.NSONE.NET
Name Server: NS-1283.AWSDNS-32.ORG
Name Server: NS-1707.AWSDNS-21.CO.UK
Name Server: NS-421.AWSDNS-52.COM
Name Server: NS-520.AWSDNS-01.NET
DNSSEC: unsigned
URL of the ICANN Whois Inaccuracy Complaint Form: https://www.icann.org/wicf/
>>> Last update of whois database: 2026-07-23T08:34:08Z <<<

For more information on Whois status codes, please visit https://icann.org/epp

NOTICE: The expiration date displayed in this record is the date the
registrar's sponsorship of the domain name registration in the registry is

```

Output shows:

- Domain Name: GITHUB.COM
- Registrar: **MarkMonitor Inc.** (IANA ID: 292)
- Creation Date: **2007-10-09**
- Updated Date: 2024-09-07
- Registry Expiry Date: 2026-10-09
- Domain Status: clientDeleteProhibited, clientTransferProhibited, clientUpdateProhibited
- Name Servers: DNS1-4.P08.NSONE.NET, NS-1283.AWSDNS-32.ORG, NS-1707.AWSDNS-21.CO.UK, NS-421.AWSDNS-52.COM, NS-520.AWSDNS-01.NET
- DNSSEC: unsigned

Command run: whatweb <https://google.com>

```

(Maliha@DESKTOP-JG3NK4T)-[~]
$ whatweb https://google.com
https://google.com [301 Moved Permanently] Country[UNITED STATES][US], HTTPServer[gws], IP[142.251.37.142], RedirectLocation[https://www.google.com/], Title[301 Moved], UncommonHeaders[content-security-policy-report-only,alt-svc], X-Frame-Options[SAMEORIGIN], X-XSS-Protection[0]
https://www.google.com/ [200 OK] Cookies[AEC,NID,__Secure-STRP], Country[UNITED STATES][US], HTML5, HTTPServer[gws], HttpOnly[AEC,NID], IP[142.251.157.119], Script, Title[Google], UncommonHeaders[content-security-policy-report-only,accept-ch,alt-svc], X-Frame-Options[SAMEORIGIN], X-XSS-Protection[0]

```

Explanation: The bare domain redirects (301) to `www.google.com`, which returns the actual page (200 OK). The web server is identified as **gws** (Google Web Server) — Google's own proprietary server software, not a common off-the-shelf server like Apache or nginx. The response includes clickjacking protection via `X-Frame-Options: SAMEORIGIN` (page can only be framed by itself) and sets several cookies, some `HttpOnly` (inaccessible to JavaScript, reducing XSS risk). Two different IPs appear across the two requests, reflecting Google's globally load-balanced infrastructure.

Command run: `whatweb https://github.com`

```
(Maliha@DESKTOP-JG3NK4T)-[~]  
$ whatweb https://github.com  
https://github.com [200 OK] Content-Language[en-US], Cookies[_gh_sess,octo,logged_in], Country[UNITED STATES][US], Email[you@domain.com], HTML5, HTTPServer[github.com], HttpOnly[_gh_sess,logged_in], IP[20.207.73.82], Open-Graph-Protocol[object][1401488693436528], OpenSearch[/opensearch.xml], Script[application/json,module], Strict-Transport-Security[max-age=31536000; includeSubdomains; preload], Title[GitHub · Change is constant. GitHub keeps you ahead. · GitHub], UncommonHeaders[x-content-type-options,referrer-policy,content-security-policy,x-github-request-id], X-Frame-Options[deny], X-XSS-Protection[0]
```

Explanation: Unlike Google, GitHub responds directly with 200 OK (no redirect needed). The server identifies itself simply as **github.com** rather than a standard software name. GitHub enforces HSTS with `preload`, meaning compliant browsers will *never* connect over plain HTTP, even on the first visit — stronger than a typical HSTS setup. It uses `X-Frame-Options: deny` (stricter than Google's `SAMEORIGIN`), since there's no legitimate use case for embedding GitHub in an iframe. Session cookies (`_gh_sess`, `logged_in`) are `HttpOnly`, protecting them from client-side script access. The IP (20.207.73.82) falls in Microsoft Azure's range, consistent with GitHub's infrastructure since its 2018 acquisition by Microsoft.